Transformers: https://hackage.haskell.org/package/transformers

Closure under composition
TypeCompose: https://wiki.haskell.org/TypeCompose
Stackoverflow - Composing monads vs applicative functors: http://stackoverflow.com/q/13034229/1333025

Functors and applicatives are **closed under composition**
i.e. **preserved** under composition

> if (n x) and (m x) are functors, then n (m x) is a functor
> if (n x) and (m x) are applicatives, then n (m x) is an applicative

i.e.

```
<$> :: Functor m, Functor n => (a -> b) -> n (m a) -> n
(m b)                    -- note: map of (a->b), NOT (m
a -> m b)
<*> :: Applicative m, Applicative n => n (m (a -> b)) ->
n (m a) -> n (m b)      -- note: map of (a->b), NOT (m a
-> m b)

f <$> nmx    = (f<$>) <$> nmx
      -- (f<$>) :: m a -> m b

        -- i.e. use <$> on outer n to apply (f<$>) on
inner (m a)
nmf <*> nmx  = (<*>) <$> nmf <*> nmx
        -- (<*>) :: m (a -> b) -> m a -> m b

        -- i.e. use <*> on outer n to apply (<*>) on
inner (m (a ->b)) and (m a)
```

e.g.

```
-- e.g.
(<*>) <$> Just nf <*> Just nx
  = Just ((<*>) nf nx)
  = Just (nf <*> nx)                          -- apply in
inner

-- e.g.
(<*>) <$> Just [(*2),(*10)] <*> Just [1..3]  -- Just
[2,4,6,10,20,30]
([(*2),(*10)]<*>) <$> Just [1..3]            -- same
Just ([(*2),(*10)] <*> [1..3])               -- same
```

Closed because in a **structure (m a)** they only transform the **value a**
and NOT the **structure type m itself**.
Closedness was already known in the original applicative paper.
Applicative programming with effects: https://www.staff.city.ac.uk/~ross/papers/Applicative.html

Composing applicative functors The weakness of applicative functo
easier to construct from components. In particular, although only
monads are composable (Barr & Wells, 1984), the Applicative class
composition,

$$\textbf{newtype}\ (f \circ g)\ a = \mathsf{Comp}\{\mathsf{comp} :: (f\ (g\ a))\}$$

just by lifting the inner Applicative operations to the outer layer:

$$\begin{aligned}&\textbf{instance}\ (\mathsf{Applicative}\ f, \mathsf{Applicative}\ g) \Rightarrow \mathsf{Applicative}\ (f \circ g)\ \textbf{wh}\\ &\quad \mathsf{pure}\ x = \mathsf{Comp}\ [\![(\mathsf{pure}\ x)]\!]\\ &\quad \mathsf{Comp}\ fs \circledast \mathsf{Comp}\ xs = \mathsf{Comp}\ [\![(\circledast)\ fs\ xs]\!]\end{aligned}$$

Closure of monads
Monad transformer
Wiki - Typeclassopedia: http://wiki.haskell.org/Typeclassopedia#Monad_transformers
[Wadler] combining monads: https://homepages.inf.ed.ac.uk/wadler/topics/monads.html#combining-monads

Monad are **NOT closed** under composition necessarily
i.e.
even if n * and m * are monads, n (m *) is not necessarily a monad

e.g. A failed definition

```
>>= :: Monad m, Monad n => n (m a) -> (a -> n (m b)) ->
n (m b)
nmx >>= k =    nmx >>= (\mx ->                -- begin
processing action n
                  mx >>= (\x ->               -- begin
processing action m
                               k x))          -- NO. k
x :: n (m b) i.e. not m
```

The problem is that a monad function on n (m *) is of the form

k :: a -> n (m b) i.e. maps an **inner value** x :: a of m a to a next **outer action** k x :: n (m b)
But **inner action m** knows nothing of **outer action n** so m cannot process k x
If k x is not processed by inner action m, then it never reaches the outer action n either.

Monads are not closed **in general**, but only for **specific cases.**
A **monad transformer** is such a special case.
e.g. MaybeT

```
-- MaybeT
>>= :: Monad m => m (Maybe a) -> (a -> m (Maybe b)) -> m
(Maybe b)
nmx >>= k =    nmx >>= (\mx ->
                    -- mx >>= k
-- NO. Failed general definition.
```

```
                    case mx of                          --
OK. manually compute m=Maybe
                      Just x -> k x
                      Nothing -> return Nothing)
```

Proof: a monad not closed under composition
Stackoverflow - proof that monads are not closed: http://stackoverflow.com/q/13034229/1333025

Pair and Maybe are monads, but the composition Bad a is NOT a monad

```
data Pair a = P { p1::a, p2::a }


instance Monad Pair where                    -- i.e.
termwise map
P x y >>= k = P (p1.k $ x) (p2.k $ y)

newtype Bad a = B (Maybe (Pair a))           -- not
monad
```

The problem is what to do if:

input = Just(P x y) ==> k x ↦ success=Just ==> output = Just (P *) or Nothing?
k y ↦ failure=Nothing

Both lead to contradictions.

==========

Monad Transformer Classes (MTL)

mtl: https://hackage.haskell.org/package/mtl-2.3
transformers: https://hackage.haskell.org/package/transformers-0.6.0.4

Monad Transformer classes (MTL)
Monad stack
In practice, we often need to combine **multiple functionalities**
i.e. **multiple composition** of monad transforms resulting in a **deep monad stack**
e.g. combine app configs + logs + result + Stdio
That is, combine ReaderT + WriterT + StateT + IO

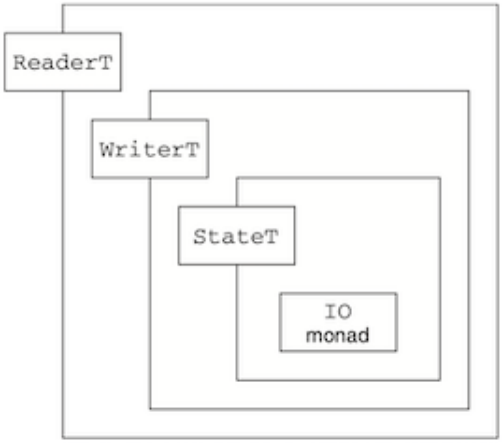


Figure 6.6 A four-level-deep monad stack based on the IO monad

Problems

Problem 1) difficult to access the **inner layers** since the functions need to be **lifted**.
e.g.

```
type MyApp0 r w s e = ReaderT r (WriterT [w] (StateT s
(ExceptT e IO)))

task :: MyApp0 Bool Int String String Bool
task = do
  x <- ask                                    --
level1: ReaderT
  lift $ tell [0]                             --
level2: WriterT
  lift $ lift $ put "Hello"                   --
level3: StateT
  _ <- lift $ lift $ lift $ throwError "Oops"   --
level4: ExceptT
  lift $ lift $ lift $ lift $ putStr "Hello"    --
level5: IO
  return x
```

Problem 2) code is coupled with the **order** of the stack
e.g.

```
-- type MyApp0 = ReaderT Bool (WriterT String)
task :: MyApp0
task = do
  lift $ tell [0]                             -- OK

-- type MyApp0 r w = WriterT String (ReaderT Bool)
task :: MyApp0
task = do
  lift $ tell [0]                             --
BROKEN
```

Solution

Abstract the **monad transformers** as **monad transformer classes**.
e.g. type **WriterT** abstracted as **MonadWriter** class
e.g. type **ExceptT** abstracted as **MonadError** class

Each concrete monad transformer **propagates** the class
e.g. MonadError implemented by ExceptT, ReaderT, WriterT, StateT

```
instance Monad m => MonadError e (ExceptT m)
instance MonadError e m => MonadError e (ReaderT r m)
instance (Monoid w, MonadError e m) => MonadError e
(WriterT w m)
instance MonadError e m => MonadError e (StateT s m)
```

Thus monad transform functions can be accessed from **any level**.
e.g. compose ReaderT + WriterT + StateT + ExceptT + IO

```
-- MyApp0 = (MonadReader, MonadWriter, MonadState,
MonadError, MonadIO)
type MyApp0 r w s e = ReaderT r (WriterT [w] (StateT s
(ExceptT e IO)))
```

MyApp0 preserves **transformers** via the classes

```
-- https://hackage.haskell.org/package/mtl-2.3/docs/
Control-Monad-Error-Class.html
-- MyApp preserves MonadError since:
   MonadError e (ExceptT e IO)
=> MonadError e (StateT s (ExceptT e IO))
                since MonadError e m => MonadError e
(StateT s m)
=> MonadError e (WriterT [w] (StateT s (ExceptT e
IO)))                 since (Monoid w, MonadError e m)
=> MonadError e (WriterT w m)
=> MonadError e (ReaderT r (WriterT [w] (StateT s
(ExceptT e IO))))     since MonadError e m => MonadError
e (ReaderT r m)
=> MonadError e MyApp0

-- https://hackage.haskell.org/package/mtl-2.3/docs/
Control-Monad-State-Lazy.html
-- MyApp reserves MonadState since:
   MonadState s (StateT s (ExceptT e IO))
=> MonadState s (WriterT [w] (StateT s (ExceptT e
IO)))                  since (Monoid w, MonadState s m)
=> MonadState s (WriterT w m)
=> MonadState s (ReaderT r (WriterT [w] (StateT s
(ExceptT e IO))))      since MonadState s m =>
MonadState s (ReaderT r m)
```

```
=> MonadState s MyApp0

-- https://hackage.haskell.org/package/mtl-2.3/docs/
Control-Monad-Writer-Class.html
-- MyApp preserves MonadWriter since:
   MonadWriter [w] (WriterT [w] (StateT s (ExceptT e
IO))
=> MonadWriter [w] (ReaderT (WriterT [w] (StateT s
(ExceptT e IO))))    since MonadWriter w m =>
MonadWriter w (ReaderT r m)
=> MonadWriter [w] MyApp0


...  similar with MonadError, MonadIO
```

Thus MyApp0 preserves **functions** of all transformers
i.e. can use MonadReader.ask, MonadWriter.tell, MonadState.put, etc

```
-- GOOD: with transformer class

taskWithTransformLift :: MyApp0 Bool Int String String
Bool
taskWithTransformLift = do
  x <- ask                      -- MonadReader.ask
  tell [0]                      -- MonadWriter.tell
  put "Hello"                   -- MonadState.put
  _ <- throwError "Oops"        -- MonadError.throwError
  liftIO $ putStrLn "Hello"     -- MonadIO.liftIO
  return x


-- without transformer class
-- i.e. manual lifting
taskWithManualLift :: MyApp0 Bool Int String String Bool
taskWithManualLift = do
  x <- ask                                    --
level1: ReaderT
  lift $ tell [0]                             --
level2: WriterT
  lift $ lift $ put "Hello"                   --
level3: StateT
  _ <- lift $ lift $ lift $ throwError "Oops"   --
level4: ExceptT
  lift $ lift $ lift $ lift $ putStr "Hello"    --
level5: IO
  return x
```

transform classes **decouples** code from the monad stack **layers**
i.e. decouples from the number and order of stack
e.g.

```
{-# LANGUAGE FlexibleContexts #-}
```

```haskell
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

import Control.Monad.Except
import Control.Monad.Reader
import Control.Monad.Writer


newtype App0 r w a = App0 {runApp0 :: ReaderT r (Writer
[w]) a}
  deriving
    ( Functor,
      Applicative,
      Monad,
      MonadWriter [w],
      MonadReader r
    )


-- same as App0 but different order
newtype App0V1 r w a = App0V1 {runApp0v1 :: WriterT [w]
(Reader r) a}
  deriving
    ( Functor,
      Applicative,
      Monad,
      MonadWriter [w],
      MonadReader r
    )


-- same as App0 but with an extra layer
newtype App0v2 e r w a = App0v2 {runApp0v2 :: WriterT
[w] (ReaderT r (Except e)) a}
  deriving
    ( Functor,
      Applicative,
      Monad,
      MonadWriter [w],
      MonadReader r
    )

-- e.g. works only for App0
workCoupled :: App0 w w w
workCoupled = App0 $ do
  x <- ask
  lift $ tell [x]
  return x


-- e.g. works for any of App0, App0v1, App0v2
workDecoupled :: (MonadReader w m, MonadWriter [w] m) =>
m w
workDecoupled = do
  x <- ask
```

```
  tell [x]
  return x

-- use
x = workCoupled @String
x = workDecoupled @String @(App0 String String)
 -- with App0
x = workDecoupled @String @(App0V1 String String)
 -- with App0v1
x = workDecoupled @String @(App0v2 () String String)
  -- with App0v2
```

==========

OLD

In order to compose multiple functions, the functions need to be **abstracted** as **interfaces** instead of **concrete** types.
i.e. concrete monad transformers abstracted as **classes**
i.e. **monad transformer classes**
e.g. type **WriterT** abstracted as **MonadWriter** class
e.g. type **ReaderT** abstracted as **MonadReader** class

With transformer classes, **multiple compositio**n of transformers **preserves** the functionality of each transformer
i.e. a deep monad stack preserves the functionality from **inner** transformer.

e.g. compose ReaderT + WriterT + StateT + ExceptT + IO

```
-- MyApp0 = (MonadReader, MonadWriter, MonadState,
MonadError, MonadIO)
type MyApp0 r w s e = ReaderT r (WriterT [w] (StateT s
(ExceptT e IO)))
```

MyApp0 preserves **transformers** via the classes

```
-- https://hackage.haskell.org/package/mtl-2.3/docs/
Control-Monad-Error-Class.html
-- MyApp preserves MonadError since:
   MonadError e (ExceptT e IO)
=> MonadError e (StateT s (ExceptT e IO))
                since MonadError e m => MonadError e
(StateT s m)
=> MonadError e (WriterT [w] (StateT s (ExceptT e
IO)))                 since MonadError e m => MonadError
e (WriterT w m)
=> MonadError e (ReaderT r (WriterT [w] (StateT s
(ExceptT e IO))))     since MonadError e m => MonadError
e (ReaderT r m)
=> MonadError e MyApp0
```

```
-- https://hackage.haskell.org/package/mtl-2.3/docs/
Control-Monad-State-Lazy.html
-- MyApp reserves MonadState since:
   MonadState s (StateT s (ExceptT e IO))
=> MonadState s (WriterT [w] (StateT s (ExceptT e
IO)))                  since MonadState s m =>
MonadState s (WriterT w m)
=> MonadState s (ReaderT r (WriterT [w] (StateT s
(ExceptT e IO))))      since MonadState s m =>
MonadState s (ReaderT r m)
=> MonadState s MyApp0

-- https://hackage.haskell.org/package/mtl-2.3/docs/
Control-Monad-Writer-Class.html
-- MyApp preserves MonadWriter since:
   MonadWriter [w] (WriterT [w] (StateT s (ExceptT e
IO))
=> MonadWriter [w] (ReaderT (WriterT [w] (StateT s
(ExceptT e IO))))    since MonadWriter w m =>
MonadWriter w (ReaderT r m)
=> MonadWriter [w] MyApp0


...  similar with MonadError, MonadIO
```

Thus MyApp0 preserves **functions** of all transformers
i.e. can use MonadReader.ask, MonadWriter.tell, MonadState.put, etc

```
-- GOOD: with transformer class

taskWithTransformLift :: MyApp0 Bool Int String String
Bool
taskWithTransformLift = do
  x <- ask                      -- MonadReader.ask
  tell [0]                      -- MonadWriter.tell
  put "Hello"                   -- MonadState.put
  _ <- throwError "Oops"        -- MonadError.throwError
  liftIO $ putStrLn "Hello"     -- MonadIO.liftIO
  return x


-- without transformer class
-- i.e. manual lifting
taskWithManualLift :: MyApp0 Bool Int String String Bool
taskWithManualLift = do
  x <- ask                                    --
level1: ReaderT
  lift $ tell [0]                             --
level2: WriterT
  lift $ lift $ put "Hello"                   --
level3: StateT
  _ <- lift $ lift $ lift $ throwError "Oops"   --
level4: ExceptT
```

```
  lift $ lift $ lift $ lift $ putStr "Hello"    --
level5: IO
  return x
```